

Universidad de San Carlos de Guatemala.
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Ingeniería en Ciencias y Sistemas
Ing. Mario Tobar



Proyecto de Seminario de Sistemas 1

Kei Beauty

Segunda Fase

Sergio Alejandro Rodriguez López

202131443

Jorge Anibal Bravo Rodríguez

202131782

Quetzaltenango 21 de septiembre del 2026

Índice

Índice.....	1
Stack tecnológico.....	3
4.1. Backend.....	3
4.2. Frontend.....	4
4.3. Entorno de Contenedores e Infraestructura.....	5
Diagrama de clases.....	6
Infraestructura Finalizada y Entorno de Despliegue.....	7
1. Topología del Sistema y Flujo de Red.....	7
2. Estado de Servicios y Contenedores.....	8
3. Estrategia de Despliegue de Origen Único (Modo Actual).....	9
4. Configuración de Red y Políticas CORS.....	9
5. Gestión de Variables y Secretos.....	10
6. Arquitectura de Producción Independiente (Disponible).....	10
Diagrama de Arquitectura.....	11
Modelo Entidad-Relación.....	12
Estructura de paquetes.....	13
Arquitectura de Paqueterías del Backend (KeiBeauty).....	13
Arquitectura de Paqueterías del Frontend.....	18
Modelos de procesos técnicos completos.....	21
Historias de usuario y casos de uso finalizadas.....	22
1. Historias de usuario.....	22
HU-01: Registro de cliente.....	22
HU-02: Inicio de sesión.....	22
HU-03: Verificación 2FA por email.....	23
HU-04: Activar/desactivar 2FA.....	23
HU-05: Recuperación de contraseña.....	24
HU-06: Restablecer contraseña.....	24
HU-07: Catálogo con filtros y orden.....	25
HU-08: Detalle de producto con galería.....	25
HU-09: Carrito de invitado.....	26
HU-10: Carrito autenticado.....	26
HU-11: Checkout y pedido.....	27
HU-12: Historial y detalle de pedidos.....	27
HU-13: Favoritos.....	28
HU-14: Alerta de stock "Avísame".....	28
HU-15: Reseñas y calificaciones.....	29

HU-16: CRUD de productos (admin).....	29
HU-17: CRUD de marcas y categorías (admin).....	30
HU-18: Inventario y reajustes (admin).....	30
HU-19: Gestión de pedidos y guías (admin).....	31
HU-20: Notificaciones in-app.....	31
HU-21: Reportes con Excel (admin).....	32
HU-22: Perfil de usuario.....	32
HU-23: Página informativa y configuración de API.....	33
2. Casos de Uso.....	34
CU-01: Registrar Usuario.....	34
CU-02: Iniciar Sesión.....	34
CU-03: Ver Catálogo de Productos.....	35
CU-04: Ver Detalle de Producto.....	36
CU-05: Gestionar Carrito de Compras.....	36
CU-06: Realizar Pedido / Checkout.....	37
CU-07: Ver Historial y Detalle de Pedidos.....	37
CU-08: Gestionar Productos - CRUD Admin.....	38
CU-09: Gestionar Estado de Pedidos y Guías.....	38
CU-10: Gestionar Inventario y Reajustes.....	39
CU-11: Consultar y Exportar Reportes.....	39
CU-12: Gestionar Productos Favoritos.....	40
CU-13: Contacto vía WhatsApp e Información Institucional.....	40
CU-14: Validar Autenticación de Dos Factores (2FA).....	41
CU-15: Recuperar y Restablecer Contraseña.....	41
CU-16: Suscribirse a Alertas de Stock ("Avísame").....	42
CU-17: Registrar y Consultar Reseñas de Productos.....	42
CU-18: Gestionar Marcas y Categorías - CRUD Admin.....	43
CU-19: Consultar Notificaciones In-App.....	43
CU-20: Gestionar Perfil de Usuario.....	44
Repositorio de GitHub.....	45

Stack tecnológico

Para el desarrollo y despliegue de la plataforma web Kei Beauty, se seleccionó una arquitectura desacoplada basada en una API RESTful en el Backend y una Single Page Application (SPA) en el Frontend. A continuación, se detalla el conjunto de tecnologías, librerías, versiones y la función de cada una dentro del ecosistema del sistema.

4.1. Backend

El backend está desarrollado sobre una arquitectura modular utilizando Python y Flask, estructurado mediante Blueprints para separar las capas de controladores, servicios y acceso a datos.

- **Python (v3.12+)**: Lenguaje de programación principal para el backend, elegido por su legibilidad, robustez y amplio soporte de librerías.
- **Flask (v3.0.3)**: Framework web ligero para el desarrollo de la API RESTful. Permite modularizar la aplicación mediante Blueprints.
- **Flask-SQLAlchemy (v3.1.1)**: Object-Relational Mapping (ORM) para la abstracción y gestión de consultas a la base de datos relacional PostgreSQL.
- **Flask-Migrate / Alembic (v4.0.5)**: Herramienta para la gestión y control de versiones de la base de datos (15 migraciones DDL/DML versionadas y automatizadas).
- **Flask-JWT-Extended (v4.6.0)**: Gestión de autenticación y autorización mediante Tokens JWT. Maneja tokens de acceso (24h), refresh (30d) y tokens temporales de 2FA (10 min).
- **Flask-CORS (v4.0.1)**: Control de acceso de origen cruzado (CORS), configurado dinámicamente según el entorno (localhost, ngrok o dominio en producción).
- **psycopg2-binary (v2.9.9)**: Adaptador y driver de conexión nativo entre Python y la base de datos PostgreSQL 16.
- **python-dotenv (v1.0.1)**: Carga de variables de entorno desde archivos .env para proteger credenciales y claves secretas en desarrollo.

- **Gunicorn (v22.0.0):** Servidor HTTP WSGI para producción, configurado con 4 workers asíncronos en contenedor bajo usuario no privilegiado (appuser).
- **imagekitio (v5.9.0):** SDK para la gestión, optimización y almacenamiento en la nube de imágenes (productos, marcas y guías de envío).
- **openpyxl (v3.1.5):** Librería para la generación y exportación automatizada de reportes del panel administrativo en formato Excel (.xlsx).
- **ZohoMail SMTP:** Servicio SMTP para el envío en tiempo real de correos electrónicos transaccionales (2FA, recuperación de contraseña, confirmación y estados de pedido).

4.2. Frontend

El frontend se desarrolló como una SPA orientada a la reactividad y la experiencia de usuario), utilizando la paleta de colores corporativa de Kei Beauty (primario oliva #8CB07A, tipografías Playfair Display e Inter).

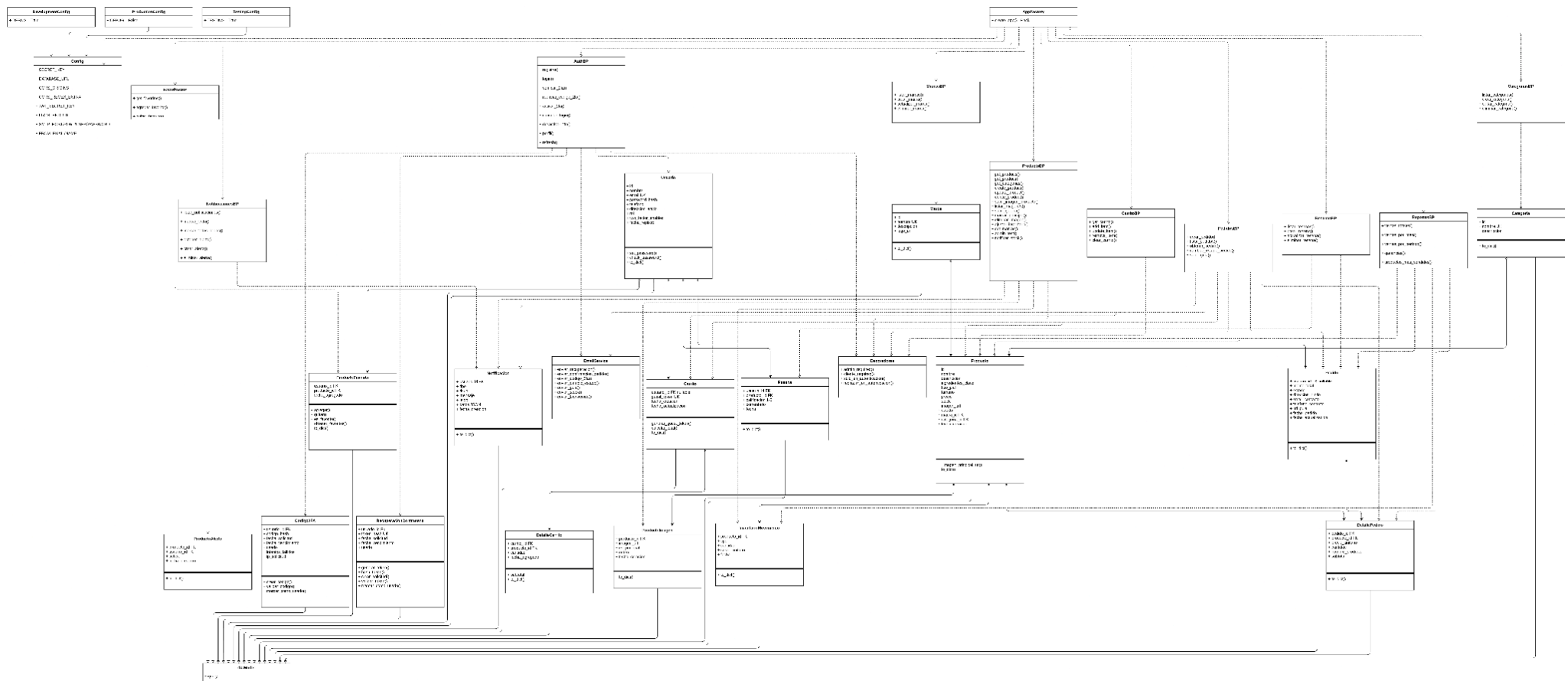
- **Vue 3 (v3.4.0):** Framework progresivo en JavaScript utilizando Composition API () para la construcción reactiva de 24 vistas del sistema.
- **Vite (v5.0.0):** Empaquetador y entorno de desarrollo ultra rápido para servir la aplicación (puerto 5173) y realizar los builds optimizados de producción.
- **Vue Router (v4.3.0):** Enrutador oficial para la gestión de 27 rutas dinámicas en la SPA, con guards de navegación para control de autenticación, 2FA y accesos como invitado.
- **Pinia (v2.1.0):** Gestor de estado centralizado. Administra 4 stores globales: auth (sesión), carrito (compras), favoritos y notificaciones.
- **Axios (v1.6.0):** Cliente HTTP basado en promesas con interceptores globales para inyección de tokens JWT y manejo uniforme de errores de servidor.
- **Bootstrap (v5.3.8):** Framework de diseño CSS/JS responsivo para maquetación, componentes colapsables y menús desplegables.

- **Bootstrap Icons (v1.13.1):** Librería de iconos vectoriales para la interfaz de usuario, manteniendo una apariencia limpia e institucional.
- **vue-toastification (v2.0.0-rc.5):** Sistema de notificaciones emergentes (toasts) en pantalla para feedback en tiempo real al usuario.
- **Nginx (alpine):** Servidor web liviano para el empaquetado final del cliente estático, configurado con fallback SPA para la gestión de rutas de Vue Router.
- **Ngrok (v3):** Herramienta de tunelización segura utilizada para exponer el entorno local y realizar pruebas móviles y remotas.

4.3. Entorno de Contenedores e Infraestructura

- **Contenedorización:** Implementada mediante Docker y Docker Compose, separando los entornos de la API Backend (api) y la Base de Datos (db con postgres:16-alpine), asegurando compilaciones multi-etapa (multi-stage builds) ligeras y aisladas.
- **Infraestructura Cloud:** Despliegue futuro en AWS (S3/CloudFront para el Frontend estático y AWS ECS/EC2 para los contenedores Docker del Backend), garantizando alta disponibilidad, seguridad por HTTPS y escalabilidad.

Diagrama de clases



Infraestructura Finalizada y Entorno de Despliegue

A continuación se detalla el estado real y verificado de la infraestructura local, de contenedores y de pruebas móviles utilizada para el proyecto Kei Beauty.

1. Topología del Sistema y Flujo de Red

El sistema opera bajo un entorno contenedorizado con Docker Compose que integra el backend y la base de datos, expuesto hacia dispositivos externos y móviles mediante tunelización segura con ngrok para pruebas:

Clientes Externos (Móviles y PCs en red externa): Acceden vía conexión HTTPS a través de la URL pública asignada por ngrok (puerto 5000 expuesto).

Entorno de Desarrollo (PC Linux local):

- Ngrok: Mantiene un túnel HTTP único redirigido hacia el puerto 5000.
- Entorno de Desarrollo Frontend: Servidor de desarrollo Vite en puerto 5173.
- Docker Compose (Red interna):
 - **Contenedor keibeauty_api (puerto 5000):** Servidor de aplicación Flask administrado por Gunicorn con variable STATIC_DIR activa.
 - **Contenedor keibeauty_db (puerto 5432):** Base de datos PostgreSQL 16 Alpine en estado saludable.
 - **Volumen Montado:** Enlace de lectura desde el build del frontend (/dist) hacia el directorio del contenedor (/frontend-dist).
 - **Servicios Externos Integrados:** Conexión saliente directa hacia el servidor SMTP de ZohoMail y el CDN de ImageKit.

2. Estado de Servicios y Contenedores

Servicio keibeaity_api:

- Tecnología: Flask 3.0 y Gunicorn (configurado con 4 workers asíncronos y bandera de recarga).
- Puerto Interno: 5000.
- Estado: Operativo (Up).
- Verificación: Respuesta HTTP 200 al consultar el endpoint de salud /health indicando estado de servicio correcto.

Servicio keibeaity_db:

- Tecnología: PostgreSQL versión 16 Alpine.
- Puerto Interno: 5432.
- Estado: Saludable (Healthy).
- Verificación: Prueba de conexión pg_isready superada y migraciones de esquemas al día.

Servidor de Desarrollo Frontend:

- Tecnología: Vite versión 5.4.21.
- Puerto Interno: 5173.
- Estado: Operativo (Up).
- Verificación: Respuesta HTTP 200 en la ruta raíz con redirección al catálogo.

Túnel de Comunicación ngrok:

- Tecnología: ngrok versión 3.
- Estado: En línea (Online).
- Verificación: Acceso público verificado en los endpoints /health y /config-api con respuesta HTTP 200.

Estado de la Base de Datos:

Esquema compuesto por 16 tablas relacionales, versión de migración Alembic en 2026_inventario_costo_unitario y datos iniciales cargados (1 usuario administrador, 3 marcas, 3 categorías y 5 productos).

3. Estrategia de Despliegue de Origen Único (Modo Actual)

Para simplificar las pruebas y optimizar el consumo de recursos durante el desarrollo, el sistema se configuró para servirse bajo un solo origen:

- **Proceso de Compilación:** El comando de compilación del frontend genera los archivos estáticos en la carpeta dist.
- **Montaje de Volúmenes:** El archivo docker-compose.yml vincula la carpeta compilada al contenedor de la API en modo de solo lectura.
- **Servidor Único de Contenido:** Al activar la variable `STATIC_DIR` en el entorno, la API en Flask se encarga de servir la aplicación SPA en la ruta raíz y administrar las rutas dinámicas del cliente, mientras que los endpoints de datos y salud continúan respondiendo en formato JSON.
- **Ventajas Operativas:** Permite utilizar un único túnel de ngrok, elimina problemas de política de origen cruzado (CORS) al compartir el mismo dominio y simplifica el despliegue a un solo servicio activo.

4. Configuración de Red y Políticas CORS

- **Entorno Local:** Expresiones regulares que permiten conexiones desde cualquier puerto en direcciones de bucle local (localhost y 127.0.0.1).
- **Entorno Móvil y ngrok:** Inclusión de patrones de origen en las variables de entorno para habilitar dominios de ngrok, permitiendo y enviando la cabecera de omisión de advertencia del navegador en las peticiones Axios.
- **Anfitriones Permitidos en Vite:** Configuración explícita de dominios autorizados para localhost, bucles locales y subdominios de ngrok.

5. Gestión de Variables y Secretos

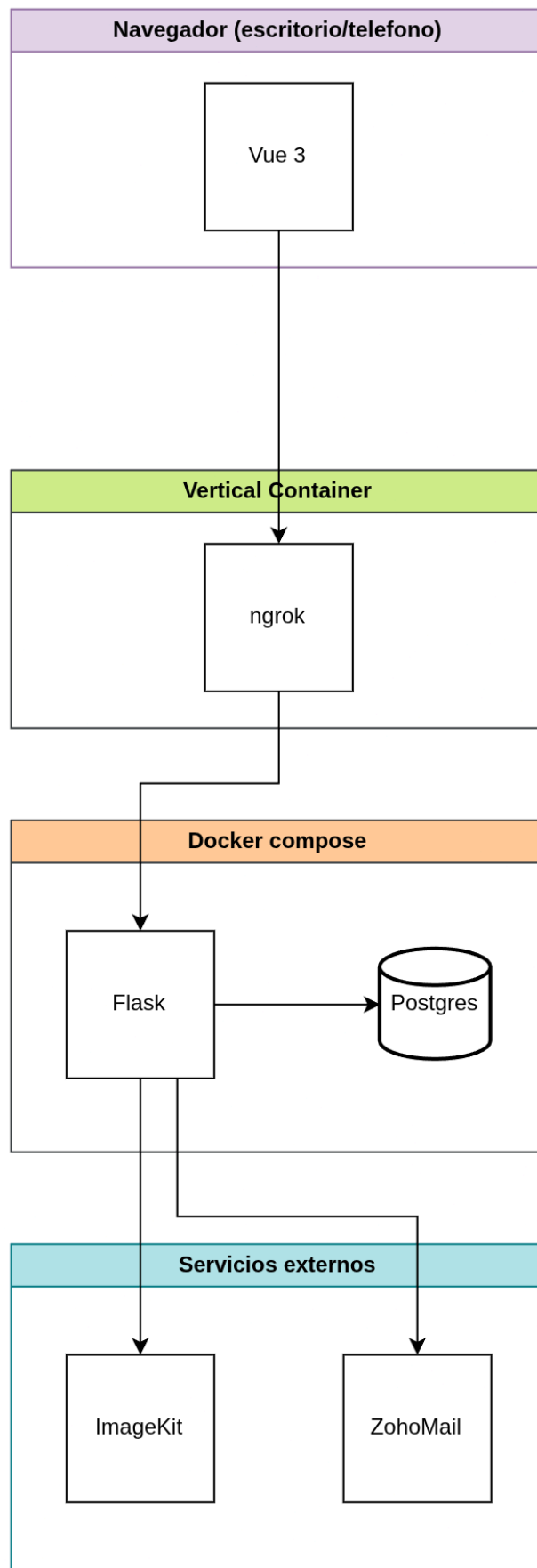
- **Archivo de Entorno Backend (.env):** Mantiene fuera del control de versiones las claves secretas de la aplicación, claves para tokens JWT, cadena de conexión a PostgreSQL, variables de puerto y origen CORS, ruta del directorio estático, credenciales SMTP para ZohoMail y claves API de ImageKit.
- **Archivo de Entorno Frontend (.env):** Configura la URL base por defecto de la API y el enlace hacia la cuenta de atención de WhatsApp.
- **Ajuste de API Dinámico:** Permite redefinir la dirección del servidor backend en tiempo de ejecución desde el navegador mediante la pantalla de configuración, almacenando el valor en la memoria local del cliente.

6. Arquitectura de Producción Independiente (Disponible)

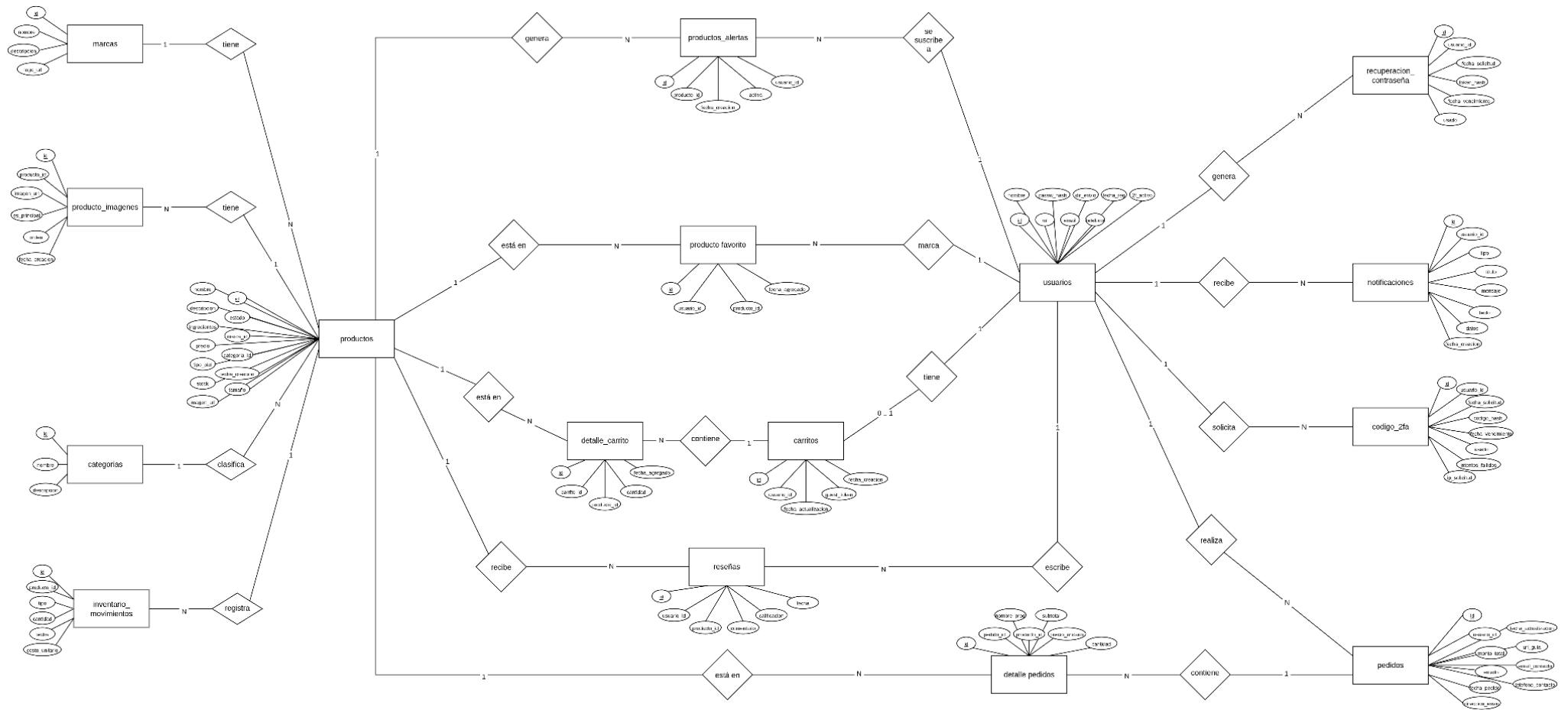
El proyecto cuenta con la preparación técnica para migrarse a un entorno de producción desacoplado:

- **Frontend:** Archivo Dockerfile listo para compilación multi-etapa con servidor Nginx optimizado y reglas de reescritura para rutas de SPA.
- **Backend:** Imagen Docker optimizada ejecutada con el usuario sin privilegios appuser sobre Gunicorn, preparada para vincularse a una instancia de PostgreSQL administrada en la nube, con entorno de producción forzado, credenciales reales y políticas estricta de seguridad HTTPS.

Diagrama de Arquitectura



Modelo Entidad-Relación



Estructura de paquetes

El proyecto Kei Beauty utiliza un enfoque de arquitectura desacoplada, distribuido en dos repositorios de código independientes: KeiBeauty/ para el Backend y KeiBeauty-frontend/ para el Frontend.

A continuación, se detalla la responsabilidad de cada paquete, módulo y archivo clave en ambos lados.

Arquitectura de Paquetes del Backend

El servidor Backend está desarrollado en Python con el framework Flask, estructurado mediante el patrón Factory y modularizado en controladores (Blueprints), modelos ORM (SQLAlchemy), utilidades de correo y autorización, y un control estricto de migraciones de la base de datos.

Raíz del Proyecto:

- **app.py:** Archivo de entrada (Factory Pattern via `create_app()`). Configura las extensiones CORS y JWT, inicializa la base de datos, registra los 10 Blueprints del sistema, expone el endpoint de salud (`/health`) y sirve los archivos de la SPA estática si `STATIC_DIR` está habilitado.
- **config.py:** Clases de configuración por entorno (`DevelopmentConfig`, `ProductionConfig`, `TestingConfig`) que leen dinámicamente las variables de entorno (`.env`).
- **seed.py:** Script de inicialización de datos para entornos nuevos (`seed_data()`). Inserta el usuario administrador, 3 marcas iniciales, 3 categorías y 5 productos base.

Paquete de Modelos (models/):

Contiene 16 clases ORM mapeadas con SQLAlchemy que gestionan la persistencia, restricciones e integridad de los datos.

- **models/db.py:**
Instancia principal y compartida del objeto SQLAlchemy (db).
- **models/usuario.py:**
Clase Usuario. Gestiona credenciales (hashing pbkdf2), roles (Admin/Cliente), estado de cuenta y flag de autenticación de dos factores (2FA).
- **models/marca.py:**
Clase Marca. Almacena el catálogo de marcas y sus logotipos.
- **models/categoria.py:**
Clase Categoria. Define la clasificación de productos (Skincare, Haircare, etc.).
- **models/producto.py:**
Clase Producto. Almacena la información principal del producto y métodos de conversión to_dict() incluyendo marcas, categorías e imágenes asignadas.
- **models/producto_imagen.py:**
Clase ProductoImagen. Galería fotográfica de productos con soporte para imagen principal, orden de visualización y eliminación en cascada (CASCADE).
- **models/producto_favorito.py:**
Clase ProductoFavorito. Registro de productos marcados como favoritos por usuario, con restricción de unicidad por par (usuario, producto).
- **models/producto_alerta.py:**
Clase ProductoAlerta. Sistema de notificaciones "Avísame cuando haya stock", restringido a una única alerta activa por producto y usuario.

- **models/carrito.py:**
Clases Carrito y DetalleCarrito. Gestiona las sesiones de compra tanto para usuarios autenticados como para invitados mediante `guest_token`, calculando subtotales y totales.
- **models/pedido.py:**
Clases Pedido y DetallePedido. Almacena el historial y estado de órdenes con un snapshot histórico del precio del producto al momento de la compra.
- **models/codigo_2fa.py:**
Clase Codigo2FA. Generación, expiración (10 min) y validación de códigos de verificación numéricos de 6 dígitos.
- **models/recuperacion_contrasena.py:**
Clase RecuperacionContrasena. Tokens encriptados con SHA-256 de un solo uso con vigencia de 24 horas para reestablecimiento de credenciales.
- **models/resena.py:**
Clase Resena. Valoración (1 a 5 estrellas) y comentarios asociados exclusivamente a compras verificadas.
- **models/notificacion.py:**
Clase Notificacion. Sistema in-app con tipos de alerta y estructura de datos JSON.
- **models/inventario_movimiento.py:**
Clase InventarioMovimiento. Registro de trazabilidad para entradas, salidas, costo unitario y reajustes manuales de stock.

Paquete de Controladores y Rutas (routes/):

Módulos que exponen los 10 Blueprints de la API REST (/api/...). Se encargan de la validación de entrada, ejecución de la lógica de negocio y respuesta en formato JSON.

- **routes/auth.py (/api/auth):**
Gestión de registro, inicio de sesión, verificación 2FA, gestión de perfil, refresco de tokens JWT y recuperación de contraseñas.
- **routes/products.py (/api/products):**
Operaciones CRUD de productos, filtros avanzados, galería de imágenes y control de inventario.
- **routes/categorias.py (/api/categorias):**
Operaciones CRUD para la administración de categorías.
- **routes/marcas.py (/api/marcas):**
Operaciones CRUD para el catálogo de marcas y carga de logos.
- **routes/carrito.py (/api/carrito):**
Gestión del carrito de compras sincronizado para usuarios autenticados e invitados.
- **routes/pedidos.py (/api/pedidos):**
Flujo de checkout, actualización de estados de envío y asignación de guías de rastreo.
- **routes/favoritos.py (/api/favoritos):**
Endpoints para agregar, eliminar y listar productos favoritos.
- **routes/resenas.py (/api/resenas):**
Creación y lectura de valoraciones sobre productos adquiridos.
- **routes/notificaciones.py (/api/notificaciones):**
Notificaciones in-app y gestión de alertas automáticas por stock bajo.
- **routes/reportes.py (/api/reportes):**
Generación de reportes de negocio en formato JSON y exportación automatizada a archivos de Excel (.xlsx).

Paquete de Utilidades (utils/):

- **utils/decorators.py:**

Decoradores personalizados (admin_required, cliente_required, solo_en_autenticacion, rechazar_en_autenticacion) para el control de acceso y autorización por roles.

- **utils/email.py:**

Clase EmailService que centraliza 9 métodos para envíos de correo transaccionales por SMTP (bienvenida, 2FA, recuperación, confirmación de pedido, guías de envío y alertas de stock).

Control de Versiones de Base de Datos (migrations/):

- **migrations/versions/:** Almacena 15 scripts de migración DDL/DML que garantizan la evolución versionada del esquema relacional (versión actual: 2026_inventario_costo_unitario).

Arquitectura de Paquetes del Frontend

El cliente web se desarrolló sobre Vue 3 utilizando Vite como empaquetador, estructurado modularmente en servicios de API, almacenes de estado (Pinia), hojas de estilo globales y 24 vistas individuales.

Archivos Principales y Configuración:

- **main.js:** Punto de entrada de la aplicación. Inicializa la app de Vue, la instancia de Pinia, el enrutador Vue Router, el sistema de Toasts, carga la paleta de colores y ejecuta la verificación de sesión inicial (initAuth).
- **App.vue:** Componente layout principal. Renderiza la barra de navegación (Navbar con logo, enlaces, ícono del carrito, centro de notificaciones y avatar), la vista dinámica activa, el pie de página (Footer) y el botón flotante de WhatsApp.

Estilos y Recursos (assets/):

- **assets/estilos/paleta.css:** Definición de variables CSS corporativas (--kei-*) y sobreescritura de componentes de Bootstrap para mantener consistencia visual.
- **assets/kei-beauty.jpg:** Recurso gráfico del logotipo de la marca utilizado en el encabezado e ícono de la aplicación.

Enrutamiento (router/):

- **router/index.js:** Define las 27 rutas dinámicas de la SPA y configura los guards de navegación para proteger vistas privadas, restringir acceso por roles (Admin/Cliente), verificar estado 2FA y manejar parámetros dinámicos de API.

Capa de Servicios de Red (services/):

- **services/api.js:** Configuración de la instancia de Axios con la URL base dinámica. Contiene ~40 funciones para el consumo ordenado de todos los endpoints de la API (autenticación, catálogo, carrito, pedidos y panel administrativo).

Gestores de Estado Centralizados (stores/):

- **stores/authStore.js:**

Administra el estado de la sesión, almacenamiento de tokens JWT, datos del perfil de usuario y manejo automático de cierre de sesión ante errores 401/422.

- **stores/cartStore.js:**

Controla la lógica del carrito de compras sincronizando datos entre el servidor y la sesión de invitado (guest_token), manejando subtotales y reversión optimista.

- **stores/favoritosStore.js:**

Mantiene el listado global de productos favoritos del cliente y métodos de alternancia (toggle).

- **stores/notificacionStore.js:**

Maneja las últimas 50 notificaciones recibidas, contador de no leídas y ciclo de consulta periódica (polling) cada 30 segundos.

Capa de Vistas (views/):

Se divide funcionalmente en vistas públicas, de autenticación, de usuario y de administración:

- **Vistas Públicas:**

HomeView (página de inicio), CatalogView (catálogo con filtros), ProductDetailView (detalle del producto y reseñas), CartView (carrito), CheckoutView (proceso de pago/confirmación) y ConfigApiView (pantalla de configuración de servidor).

- **Vistas de Autenticación:**

LoginView, RegisterView, ForgotPasswordView, ResetPasswordView y TwoFactorView.

- **Vistas de Cuenta de Usuario:**

ProfileView (perfil), FavoritosView, OrderHistoryView (historial de pedidos) y OrderDetailView (detalle del pedido).

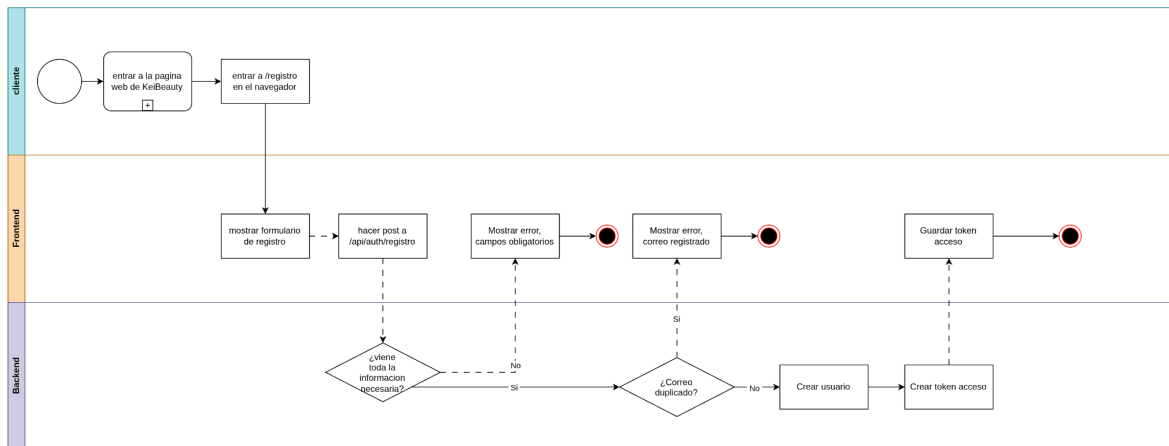
- **Vistas del Panel Administrativo:**

AdminDashboardView, ProductsView, ProductoCrearView, ProductoEditarView, MarcasView, CategoriesView, OrdersView, ResenasView y ReportesView (métricas y exportación a Excel).

Modelos de procesos técnicos completos

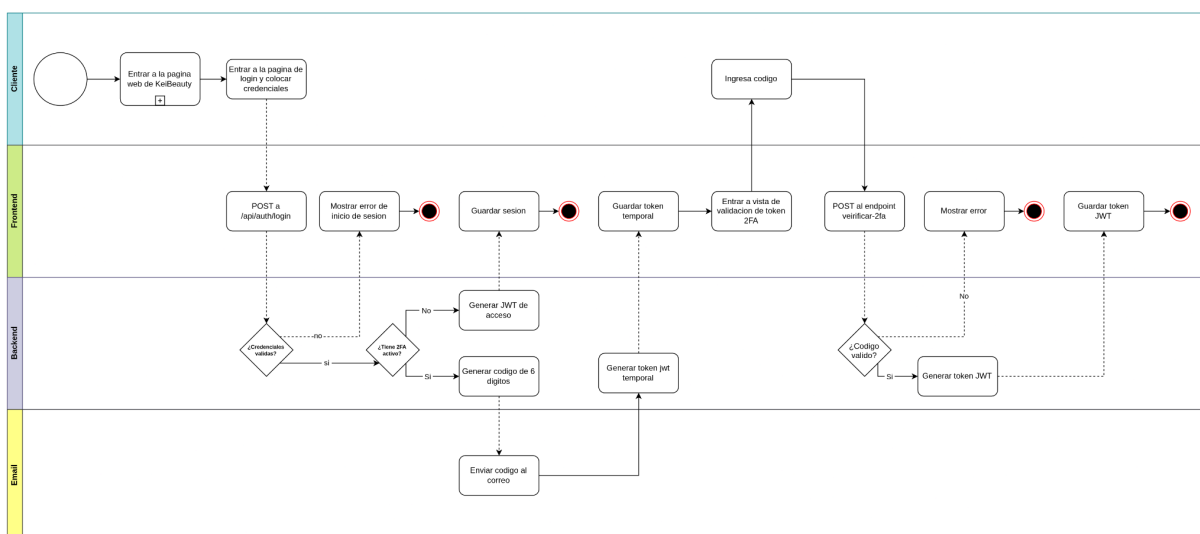
1. Registro de usuario cliente

El cliente invitado ingresa al formulario de registro en la plataforma y completa sus datos personales, de contacto y dirección de envío. El sistema valida que el correo no esté registrado previamente y, de ser válido, aplica un hash a la contraseña, crea la cuenta en la base de datos con rol de cliente y genera automáticamente los tokens de sesión para redirigirlo al catálogo.



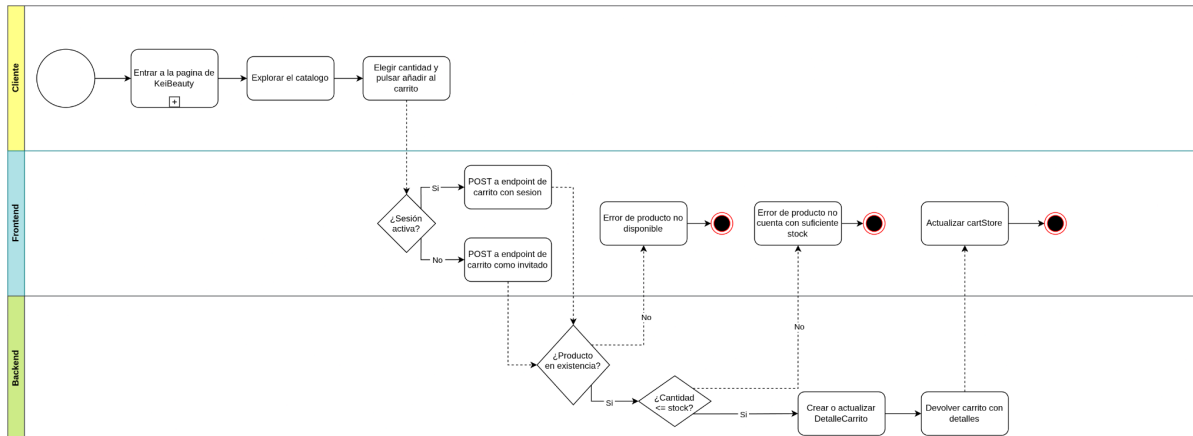
2. Iniciar sesión

El usuario ingresa sus credenciales en la pantalla de inicio de sesión. Si la contraseña es correcta y la cuenta no requiere autenticación de dos factores, el sistema entrega los tokens de acceso directamente. Si la cuenta tiene 2FA activo, el backend genera un código de 6 dígitos que envía por correo vía ZohoMail y redirige al usuario a la vista de verificación para validar el código antes de conceder la sesión.



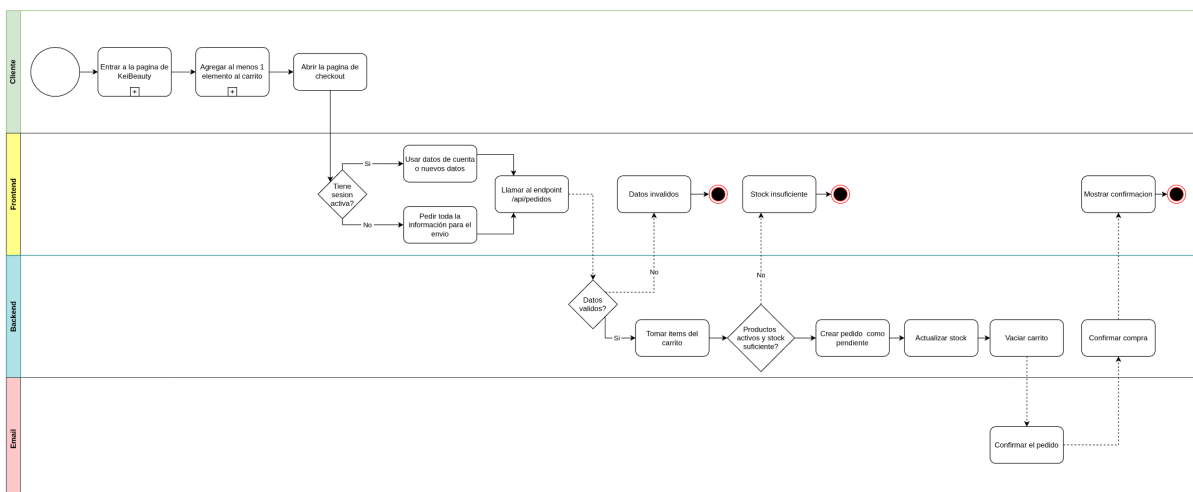
3. Agregar producto a carrito

El cliente explora el catálogo aplicando filtros por marca, categoría o búsqueda por texto. Al seleccionar un producto, el sistema muestra su ficha técnica con galería y reseñas. El usuario define la cantidad y lo agrega al carrito, momento en el que el sistema valida la disponibilidad de stock y actualiza el estado del carrito (asociado a un token de invitado o JWT de usuario registrado) en la interfaz.



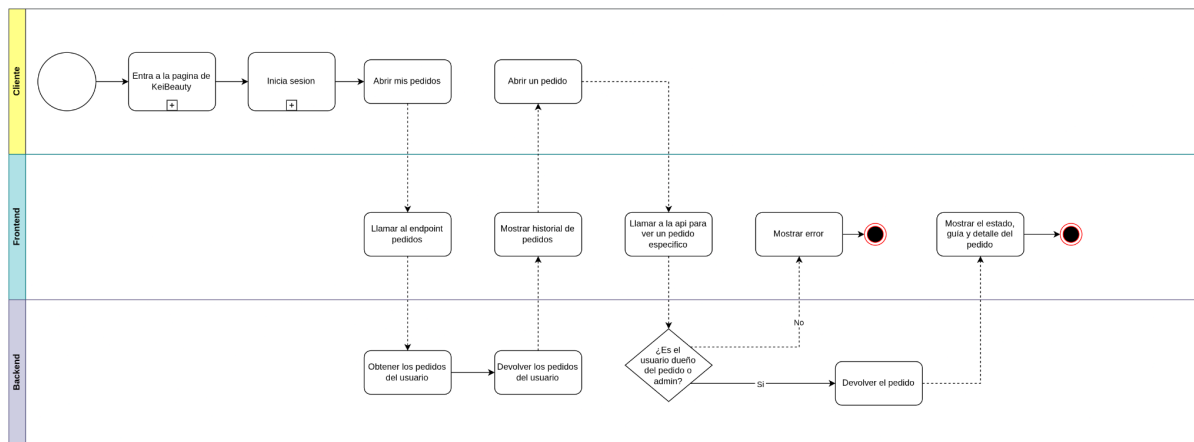
4. Checkout y confirmación de pedido

Desde el carrito, el cliente procede a confirmar el pedido con sus datos de envío y contacto. El sistema ejecuta una transacción SQL para verificar y bloquear el stock disponible de los artículos. Si hay existencias suficientes, descuenta el inventario, guarda una copia del precio actual de cada producto, vacía el carrito, registra el pedido como pendiente y envía la confirmación por correo electrónico.



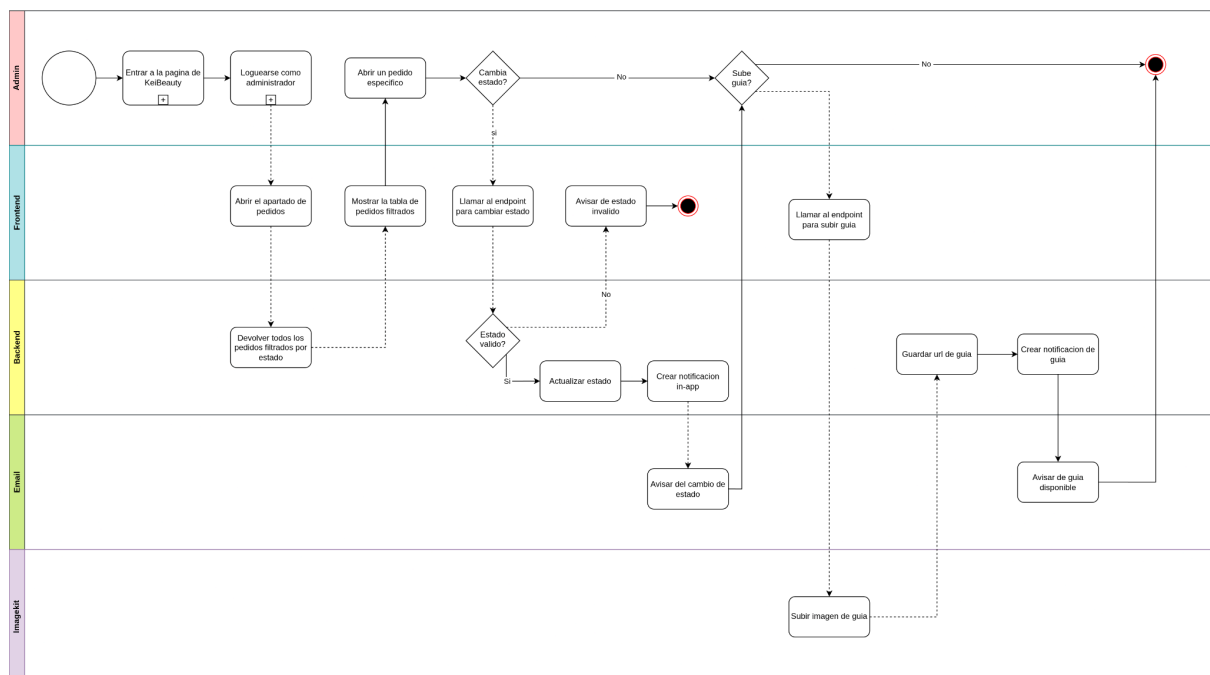
5. Consultar estado de pedido

El cliente revisa su historial en /mis-pedidos, abre el detalle para ver estado, guía y totales, y se entera de cambios por la campana de notificaciones o el email.



6. Gestión del administrador de pedidos

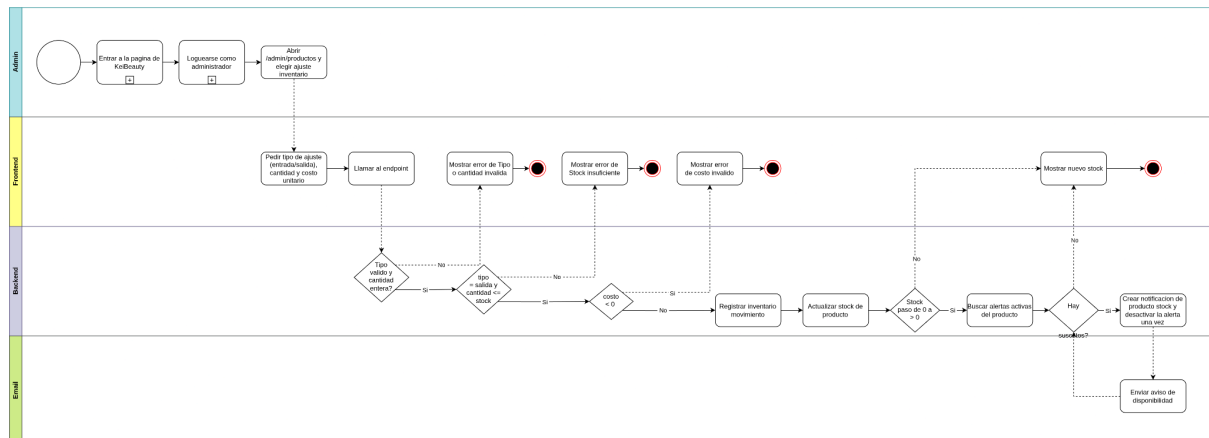
El administrador accede al panel de control para revisar el detalle de las órdenes entrantes y actualizar su estado a en preparación, enviado o entregado. Durante el proceso, el administrador puede adjuntar la imagen de la guía de transporte, la cual se sube al CDN de ImageKit para actualizar el pedido y notificar automáticamente al cliente tanto por la aplicación como por correo electrónico.



7. Administrar inventario de productos

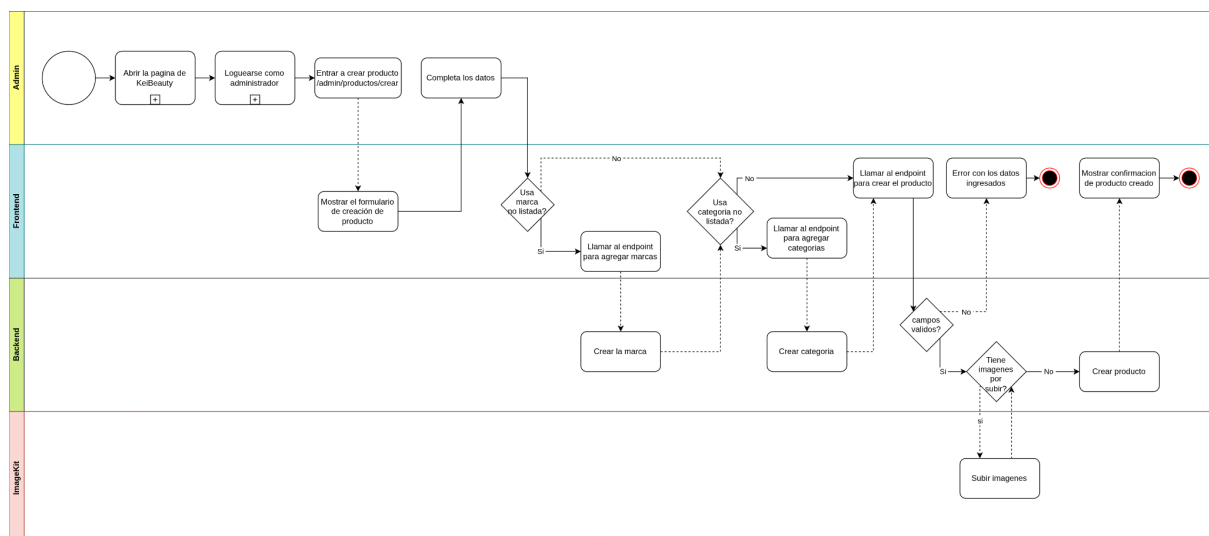
Valida el tipo de operacion (si es de entrada o salida), si es una cantidad entera y mayor a cero junto con un costo mayor o igual a cero. En caso de ser una salida con stock insuficiente se rechaza.

Al finalizar registra el movimiento de inventario, busca las alertas activas por cada suscriptor y crea sus notificaciones, al terminar las desactiva y envía correos a suscriptores de la alerta.



8. Agregar un producto nuevo

La administradora abre el formulario de crear producto y lo llena. Si la marca o la categoría no están listadas, las escribe como texto y el sistema las crea primero vía sus endpoints; luego envía el producto (multipart), el backend valida los obligatorios, sube la imagen a ImageKit, crea el producto (con estado agotado si el stock es 0), luego se muestra confirmación del producto creado.



Historias de usuario y casos de uso finalizadas

1. Historias de usuario

HU-01: Registro de cliente

Rol / Actor	Invitado
Narrativa	Como visitante, Quiero crear una cuenta con mis datos, Para comprar con carrito persistente, favoritos e historial.
Criterios de Aceptación	• Obliga los campos nombre, email, password, direccion_envio; telefono es opcional. • Retorna estado 201 con objeto usuario (rol cliente), access_token (24h) y refresh_token (30d). • Retorna estado 400 si falta algún campo obligatorio o si el email ya se encuentra registrado.
Mapeo Técnico	Backend: POST /api/auth/registro (AuthBP.registro) Frontend: RegisterView.vue + authStore.register

HU-02: Inicio de sesión

Rol / Actor	Invitado / Cliente
Narrativa	Como usuario registrado, Quiero entrar con mi email y contraseña, Para acceder a mi cuenta, carrito y pedidos.
Criterios de Aceptación	• Retorna estado 400 sin email/password; retorna 401 si las credenciales son inválidas. • Sin 2FA activo: retorna 200 con los tokens de sesión; con 2FA activo: retorna requiere_2fa junto a un token temporal (10 min). • Tras un login exitoso, se cargan automáticamente el perfil, carrito y favoritos.
Mapeo Técnico	Backend: POST /api/auth/login (AuthBP.login) Frontend: LoginView.vue + authStore.login

HU-03: Verificación 2FA por email

Rol / Actor	Cliente (en flujo 2FA)
Narrativa	Como usuario con 2FA activo, Quiero confirmar un código de 6 dígitos enviado a mi correo, Para proteger mi cuenta aunque comprometan mi contraseña.
Criterios de Aceptación	• El código es válido por 5 minutos, de un solo uso y con un máximo de 5 intentos. Al validar, retorna 200 con los tokens definitivos. • Retorna estado 400 si el código es erróneo o expiró. El reenvío se limita a 1 código cada 60 segundos (429). • El guard de navegación redirige todo intento a /verificar-2fa mientras dure el flujo.
Mapeo Técnico	Backend: POST /api/auth/verificar-2fa, /reenviar-codigo-2fa, /cancelar-login Frontend: TwoFactorView.vue + authStore.verify2FA

HU-04: Activar/desactivar 2FA

Rol / Actor	Cliente
Narrativa	Como cliente, Quiero activar o desactivar el segundo factor de autenticación desde mi perfil, Para gestionar libremente mi nivel de seguridad.
Criterios de Aceptación	• Requiere token JWT válido; responde con estado 200 y la estructura {two_factor_enabled: true false}. • Al desactivar el módulo de seguridad, se invalidan de inmediato los códigos 2FA pendientes.
Mapeo Técnico	Backend: POST /api/auth/activar-2fa, /desactivar-2fa Frontend: ProfileView.vue

HU-05: Recuperación de contraseña

Rol / Actor	Cliente
Narrativa	Como usuario que olvidó su clave, Quiero recibir un enlace de recuperación por email, Para restablecer mi acceso a la plataforma.
Criterios de Aceptación	• Retorna siempre respuesta 200 por seguridad (sin revelar si el email existe en la BD); genera un token SHA-256 válido por 24h de un solo uso. • Envía un correo electrónico mediante ZohoMail SMTP con el enlace directo al frontend.
Mapeo Técnico	Backend: POST /api/auth/olvide-contrasena Frontend: ForgotPasswordView.vue

HU-06: Restablecer contraseña

Rol / Actor	Cliente
Narrativa	Como usuario con un enlace de recuperación, Quiero establecer una contraseña nueva, Para volver a ingresar al sistema.
Criterios de Aceptación	• Exige contraseñas coincidentes con una longitud mayor o igual a 6 caracteres. Retorna 400 si el token es inválido, expiró o ya fue usado. • Responde con estado 200 e inhabilita inmediatamente el token utilizado.
Mapeo Técnico	Backend: POST /api/auth/reestablecer-contrasena Frontend: ResetPasswordView.vue

HU-07: Catálogo con filtros y orden

Rol / Actor	Invitado / Cliente
--------------------	--------------------

Narrativa	Como visitante, Quiero buscar, filtrar por categoría/marca y ordenar por precio, Para encontrar rápidamente los productos que necesito.
Criterios de Aceptación	• Soporta queries de búsqueda categoría, marca, buscar (búsqueda literal escapando %/_) y con_favorito. • Los clientes visualizan únicamente productos en estado activo; el administrador puede ver todos y filtrar por estado. • Renderiza badges de disponibilidad ("Disponible"/"Agotado"), toasts al agregar al carrito y grilla responsiva.
Mapeo Técnico	Backend: GET /api/products, /products/categorias, /products/marcas Frontend: CatalogView.vue

HU-08: Detalle de producto con galería

Rol / Actor	Invitado / Cliente
Narrativa	Como comprador, Quiero ver fotografías, stock actual, opiniones y la marca del producto, Para tomar una decisión de compra informada.
Criterios de Aceptación	• Galería con imagen principal, carrusel de miniaturas y vista modal. Retorna 404 si el producto no existe o está inactivo. • La opción "Ver más de esta marca" redirige al catálogo filtrado mediante la ruta /catalogo?marca=<id>.
Mapeo Técnico	Backend: GET /api/products/<id>, /imagenes, GET /api/resenas?producto= Frontend: ProductDetailView.vue

HU-09: Carrito de invitado

Rol / Actor	Invitado
--------------------	----------

Narrativa	Como visitante sin cuenta, Quiero almacenar productos utilizando un token temporal, Para comprar sin necesidad de registrarme de forma obligatoria.
Criterios de Aceptación	• La cabecera X-Guest-Token identifica la sesión del carrito, validando el stock real y el estado activo del producto. • Permite agregar, actualizar cantidades, eliminar ítems individuales y vaciar el carrito por completo.
Mapeo Técnico	Backend: /api/carrito... (CarritoBP) Frontend: cartStore + CartView.vue

HU-10: Carrito autenticado

Rol / Actor	Cliente
Narrativa	Como cliente, Quiero mantener mi carrito sincronizado con mi cuenta, Para no perder mis artículos al cambiar de dispositivo.
Criterios de Aceptación	• La autenticación JWT identifica el carrito del cliente. Se recarga al iniciar sesión y se vacía al cerrarla. • Aplica actualizaciones optimistas en la interfaz con reversión automática en caso de error de conexión o servidor.
Mapeo Técnico	Backend: /api/carrito... Frontend: cartStore + CartView.vue

HU-11: Checkout y pedido

Rol / Actor	Invitado / Cliente
Narrativa	Como comprador, Quiero confirmar mi dirección de entrega y datos de contacto,

	Para generar y finalizar mi pedido.
Criterios de Aceptación	• Usuario autenticado: requiere confirmar la dirección. Invitado: exige email válido y teléfono (8+ dígitos). • Retorna 201 con el resumen del pedido (snapshot del precio/subtotal) y descuenta el stock (si llega a 0 pasa a "agotado"). • Retorna 400 si el carrito está vacío o sin stock suficiente. Dispara email automático de confirmación.
Mapeo Técnico	Backend: POST /api/pedidos (PedidosBP.crear_pedido) Frontend: CheckoutView.vue

HU-12: Historial y detalle de pedidos

Rol / Actor	Cliente
Narrativa	Como cliente, Quiero revisar el listado de mis compras pasadas y sus estados, Para darles seguimiento hasta la entrega.
Criterios de Aceptación	• Acceso restringido estrictamente a pedidos propios (retorna 401/403 en accesos ajenos). Invitados consultan mediante guest_token/email. • Despliega la guía de envío, desglose de totales y el snapshot histórico de productos adquiridos.
Mapeo Técnico	Backend: GET /api/pedidos, GET /api/pedidos/<id> Frontend: OrderHistoryView.vue, OrderDetailView.vue

HU-13: Favoritos

Rol / Actor	Cliente
--------------------	---------

Narrativa	Como cliente, Quiero marcar productos como favoritos con un icono de corazón, Para encontrarlos y comprarlos fácilmente más adelante.
Criterios de Aceptación	• Requiere inicio de sesión (muestra notificación toast si es un invitado); interacción de icono instantánea con restricción única por par (usuario, producto). • Sección dedicada en /perfil/favoritos con el listado completo.
Mapeo Técnico	Backend: GET/POST/DELETE /api/favoritos Frontend: favoritosStore + CatalogView, ProductDetailView, FavoritosView

HU-14: Alerta de stock "Avísame"

Rol / Actor	Cliente
Narrativa	Como cliente interesado en un producto agotado, Quiero suscribirme a alertas de reabastecimiento, Para recibir un aviso automático cuando vuelva a estar disponible.
Criterios de Aceptación	• Permite una sola suscripción por producto y cliente; la acción tipo toggle crea o elimina la suscripción. • Al registrar nuevo stock desde el módulo administrativo, se dispara automáticamente la notificación.
Mapeo Técnico	Backend: /api/notificaciones/producto/<id>/alerta Frontend: ProductDetailView.vue

HU-15: Reseñas y calificaciones

Rol / Actor	Cliente
Narrativa	Como compradora,

	Quiero calificar de 1 a 5 estrellas y escribir una opinión del producto, Para compartir mi experiencia con otras clientas.
Criterios de Aceptación	• Reseñas públicas en la vista de detalle con promedio general y recuento total. Exige haber comprado el producto con límite de 1 reseña por usuario. • Retorna 400 en reseñas duplicadas o valoraciones fuera de rango; retorna 403 si el usuario no tiene una compra verificada del producto.
Mapeo Técnico	Backend: /api/resenas... (ResenasBP) Frontend: ProductDetailView.vue, AdminResenasView.vue

HU-16: CRUD de productos (admin)

Rol / Actor	Admin
Narrativa	Como administradora, Quiero crear, editar y eliminar productos administrando sus imágenes, Para mantener actualizado el catálogo de la tienda.
Criterios de Aceptación	• Obliga los campos nombre, precio (> 0), stock (≥ 0), marca y categoría. Procesa solicitudes multipart con imágenes (JPG/PNG/GIF/WebP) subidas a ImageKit. • Si la marca o categoría ingresada no existe, la crea automáticamente. • Acceso protegido: retorna 401/403 si el usuario no tiene rol de administrador.
Mapeo Técnico	Backend: POST/PUT/DELETE /api/products..., galería, /marcas, /categorias Frontend: AdminProductsView, AdminProductoCrearView, AdminProductoEditarView

HU-17: CRUD de marcas y categorías (admin)

Rol / Actor	Admin
--------------------	-------

Narrativa	Como administradora, Quiero gestionar el catálogo de marcas y categorías del sistema, Para clasificar y estructurar correctamente los productos.
Criterios de Aceptación	• Exige un nombre obligatorio y único. Permite adjuntar el logo por medio de una URL o subida directa de archivo. • Retorna estado 400 si se intenta registrar una marca o categoría con un nombre duplicado.
Mapeo Técnico	Backend: /api/marcas..., /api/categorias... Frontend: AdminMarcasView.vue, AdminCategoriesView.vue

HU-18: Inventario y reajustes (admin)

Rol / Actor	Admin
Narrativa	Como administradora, Quiero registrar movimientos de entrada/salida de stock con su costo, Para llevar el control del inventario y calcular las ganancias.
Criterios de Aceptación	• Recibe la estructura {tipo: entrada salida, cantidad, costo_unitario?} y valida que exista stock suficiente ante salidas. • Las compras efectuadas por los clientes en el checkout descuentan el stock automáticamente.
Mapeo Técnico	Backend: POST /api/products/<id>/inventario Frontend: AdminProductsView.vue

HU-19: Gestión de pedidos y guías (admin)

Rol / Actor	Admin
--------------------	-------

Narrativa	Como administradora, Quiero cambiar el estado de los pedidos y adjuntar guías de envío, Para informar a las clientas sobre el avance de sus paquetes.
Criterios de Aceptación	• Maneja estados válidos (pendiente, confirmado, enviado, entregado, cancelado). Cada cambio genera una notificación in-app y un email transaccional. • Carga de guías de envío mediante formulario multipart; el archivo queda visible en la vista de detalle del pedido del cliente.
Mapeo Técnico	Backend: PATCH /api/pedidos/<id>/estado, PUT POST .../guia Frontend: AdminOrdersView.vue, OrderDetailView.vue (vista admin)

HU-20: Notificaciones in-app

Rol / Actor	Cliente
Narrativa	Como cliente, Quiero recibir alertas sobre mis pedidos y stock directamente en el icono de la campana, Para enterarme de las novedades sin tener que revisar mi correo.
Criterios de Aceptación	• Muestra las últimas 50 notificaciones con un contador de no leídas; permite marcar individualmente o en lote como leídas. • Al hacer clic, navega a la vista del pedido o producto correspondiente. Realiza consultas automáticas (polling) cada 30 segundos.
Mapeo Técnico	Backend: GET/PUT /api/notificaciones... Frontend: Menú de campana en App.vue + notificacionStore

HU-21: Reportes con Excel (admin)

Rol / Actor	Admin
--------------------	-------

Narrativa	Como administradora, Quiero consultar métricas de negocio con filtros por fechas y exportación a Excel, Para analizar el rendimiento comercial y tomar decisiones estratégicas.
Criterios de Aceptación	• Dispone de 6 reportes de negocio (Totales generales, Por mes, Por periodo con rango desde/hasta obligatorio, Ganancias netas, Top productos y Top clientes). • Al incluir el parámetro excel=1, descarga un archivo .xlsx (generado con openpyxl). Excluye ventas de pedidos cancelados. No genera archivos PDF.
Mapeo Técnico	Backend: GET /api/reportes/* (ReportesBP, openpyxl) Frontend: AdminReportesView.vue

HU-22: Perfil de usuario

Rol / Actor	Cliente
Narrativa	Como cliente, Quiero consultar mis datos personales y configurar la seguridad de mi cuenta, Para mantener mi información actualizada.
Criterios de Aceptación	• Consulta la información mediante GET /api/auth/perfil enviando el token JWT. • Si el token es inválido o expiró, el sistema limpia la sesión de forma automática al recibir códigos 401 o 422.
Mapeo Técnico	Backend: GET /api/auth/perfil Frontend: ProfileView.vue + authStore

HU-23: Página informativa y configuración de API

Rol / Actor	Invitado / Cliente
--------------------	--------------------

Narrativa	Como visitante, Quiero conocer información sobre la empresa y configurar la URL de la API, Para informarme sobre la marca y probar la aplicación desde dispositivos móviles.
Criterios de Aceptación	• La ruta /sobre-nosotros presenta información institucional; la ruta raíz / redirige directamente a /catalogo. • La vista /config-api?api=<url> permite cambiar dinámicamente el endpoint del backend, guardando la configuración y probando la conexión con la ruta /health.
Mapeo Técnico	Backend: / sirve la SPA (STATIC_DIR) Frontend: HomeView.vue, ConfigApiView.vue

2. Casos de Uso

CU-01: Registrar Usuario

Actor(es)	Cliente Invitado
Descripción	Creación de cuenta de cliente con datos de contacto y dirección de envío.
Precondiciones	No tener sesión activa (ruta guest).
Flujo Básico	1. El invitado abre /registro y completa nombre, email, password, teléfono y dirección. 2. El frontend envía POST /api/auth/registro. 3. El backend valida campos obligatorios, rechaza email duplicado, aplica hash a la password y crea el usuario con rol cliente. 4. Devuelve estado 201 con el objeto usuario, access_token (24h) y refresh_token (30d). 5. El frontend almacena la sesión, carga perfil/carrito/favoritos y redirige a /catalogo.
Flujos Alternativos / Excepciones	Estado 400 por campo obligatorio faltante o email previamente registrado en la BD.
HU Asociada	HU-01

CU-02: Iniciar Sesión

Actor(es)	Cliente Invitado / Registrado
Descripción	Autenticación mediante email y contraseña, con derivación opcional a flujo 2FA.
Precondiciones	Cuenta de usuario existente; no tener sesión activa.
Flujo Básico	1. Ingresa credenciales en /login → ejecuta POST /api/auth/login. 2. Sin 2FA activo: responde 200 con tokens y objeto usuario; el frontend hidrata la sesión. 3. Con 2FA activo: responde 200 con indicador requiere_2fa, email y token temporal (10 min); continúa en CU-14.
Flujos Alternativos / Excepciones	Estado 400 por campos vacíos; estado 401 por credenciales inválidas.

HU Asociada	HU-02
--------------------	-------

CU-03: Ver Catálogo de Productos

Actor(es)	Invitado / Cliente (vista extendida para Admin)
Descripción	Exploración del catálogo de productos con búsqueda, filtros y ordenamiento.
Precondiciones	Ninguna (la ruta raíz / redirige a /catalogo).
Flujo Básico	1. GET /api/products retorna el listado de productos (para clientes: únicamente en estado activo). 2. Permite aplicar filtros por categoria, marca, buscar (búsqueda literal escapando %/_), con_favorito y ordenamiento por precio. 3. Muestra badges de Disponibilidad/Agotado, notificación toast al añadir productos y grilla responsiva.
Flujos Alternativos / Excepciones	Búsqueda sin resultados: despliega estado vacío con opción para limpiar filtros.
HU Asociada	HU-07

CU-04: Ver Detalle de Producto

Actor(es)	Invitado / Cliente
Descripción	Ficha informativa completa del producto con galería de imágenes, stock y reseñas.
Precondiciones	El producto consultado debe existir en el sistema.
Flujo Básico	1. La ruta /producto/:id solicita GET /api/products/<id>, galería de imágenes y GET /api/resenas?producto=. 2. Muestra fotografía principal, miniaturas, modal, precio, disponibilidad de stock y reseñas con promedio de calificación. 3. La opción "Ver más de esta marca" abre /catalogo?marca=<id>.
Flujos Alternativos / Excepciones	Estado 404 si el producto no existe o si está inactivo para la vista de cliente.
HU Asociada	HU-08

CU-05: Gestionar Carrito de Compras

Actor(es)	Invitado (guest_token) / Cliente (JWT)
Descripción	Permite agregar, modificar cantidades, eliminar artículos y vaciar el carrito de compras.
Precondiciones	Ninguna; el usuario invitado recibe un guest_token en localStorage.
Flujo Básico	1. Ejecuta peticiones contra /api/carrito... enviando token JWT o cabecera X-Guest-Token. 2. El backend valida la existencia del producto, estado activo y disponibilidad de stock. 3. El frontend actualiza dinámicamente los badges de conteo y revierte la interfaz ante errores de servidor.
Flujos Alternativos / Excepciones	Estado 400 por stock insuficiente; estado 404 si el producto fue eliminado.
HU Asociada	HU-09, HU-10

CU-06: Realizar Pedido / Checkout

Actor(es)	Invitado / Cliente
Descripción	Confirmación de datos de envío y finalización del proceso de compra.
Precondiciones	Contener al menos un artículo en el carrito de compras (CU-05).
Flujo Básico	1. Vista /checkout: el usuario autenticado confirma dirección; el invitado ingresa email válido y teléfono (8+ dígitos). 2. POST /api/pedidos registra el pedido en estado pendiente almacenando un snapshot del precio, subtotal y nombre del producto. 3. Descuenta automáticamente el stock (si llega a 0 pasa a agotado), vacía el carrito y dispara correo de confirmación. 4. Retorna estado 201 con los datos del pedido y redirige a la pantalla de confirmación.
Flujos Alternativos / Excepciones	Estado 400 por datos de envío inválidos, carrito vacío o falta de stock.
HU Asociada	HU-11

CU-07: Ver Historial y Detalle de Pedidos

Actor(es)	Cliente Registrado
Descripción	Seguimiento de órdenes de compra realizadas, visualizando guía de envío y desglose de totales.
Precondiciones	Sesión de usuario activa.
Flujo Básico	1. Vista /mis-pedidos consulta el historial con GET /api/pedidos (con soporte para paginación y filtro por estado). 2. Vista /mis-pedidos/:id despliega el detalle completo, guía de rastreo adjunta y snapshot de ítems adquiridos.
Flujos Alternativos / Excepciones	Estado 401/403 al intentar consultar pedidos pertenecientes a otros usuarios; invitados consultan vía guest_token/email.
HU Asociada	HU-12

CU-08: Gestionar Productos - CRUD Admin

Actor(es)	Administrador
------------------	---------------

Descripción	Mantenimiento y administración general del catálogo de productos con gestión de imágenes.
Precondiciones	Sesión activa con rol admin (validado en la vista y en el backend).
Flujo Básico	1. Vista /admin/productos lista el inventario total (incluyendo agotados e inactivos) con filtros. 2. Creación/Edición mediante formulario multipart: requiere nombre, precio (> 0), stock (≥ 0), marca y categoría (marcas/categorías no registradas se crean automáticamente). 3. Carga de fotografías hacia ImageKit (/productos) administrando galería e imagen principal.
Flujos Alternativos / Excepciones	Estado 400 por falla de validación; 404 marca/categoría no encontrada; 401/403 acceso denegado por falta de rol.
HU Asociada	HU-16

CU-09: Gestionar Estado de Pedidos y Guías

Actor(es)	Administrador
Descripción	Actualización del flujo de entrega de pedidos y carga de guías de envío notificando al cliente.
Precondiciones	Sesión activa como administrador; pedido registrado en el sistema.
Flujo Básico	1. Desde /admin/pedidos o la vista de detalle: ejecuta PATCH .../estado (soporta 5 estados válidos). 2. Carga de imagen de guía de entrega (PUT
Flujos Alternativos / Excepciones	Estado 400 por estado inválido enviado; estado 404 si el pedido no existe.
HU Asociada	HU-19

CU-10: Gestionar Inventario y Reajustes

Actor(es)	Administrador
Descripción	Registro de entradas y salidas manuales de stock asociadas a un costo unitario para trazabilidad.

Precondiciones	Sesión activa como administrador; producto existente en la BD.
Flujo Básico	1. Solicitud POST /api/products/<id>/inventario con la estructura {tipo, cantidad, costo_unitario?}. 2. Valida el tipo de movimiento y la suficiencia de existencias en salidas; registra la transacción en el historial de inventario.
Flujos Alternativos / Excepciones	Estado 400 por tipo o cantidad inválida, o por intentar registrar una salida superior al stock actual.
HU Asociada	HU-18

CU-11: Consultar y Exportar Reportes

Actor(es)	Administrador
Descripción	Visualización de métricas generales de negocio y exportación de información comercial a Excel.
Precondiciones	Sesión activa como administrador; existencia de transacciones y pedidos en el sistema.
Flujo Básico	1. La vista /admin/reportes consulta 6 endpoints analíticos aplicando filtros desde/hasta/limit. 2. Al enviar excel=1, descarga el reporte consolidado en formato .xlsx (generado vía openpyxl), excluyendo pedidos cancelados.
Flujos Alternativos / Excepciones	Los reportes por período requieren ingresar obligatoriamente las fechas desde y hasta. Únicamente genera archivos Excel (no genera PDF).
HU Asociada	HU-21

CU-12: Gestionar Productos Favoritos

Actor(es)	Cliente Registrado
Descripción	Administración de lista de deseos (wishlist) personal con marcaje instantáneo de corazón.
Precondiciones	Sesión de cliente activa (visitantes invitados reciben un aviso emergente solicitando iniciar sesión).

Flujo Básico	1. Acción tipo toggle en catálogo/detalle (POST/DELETE /api/favoritos/<id>), pintando de rojo el corazón de forma inmediata. 2. Vista /perfil/favoritos despliega la colección guardada del cliente.
Flujos Alternativos / Excepciones	Operación idempotente gracias a la restricción de unicidad por par (usuario, producto) en la base de datos.
HU Asociada	HU-13

CU-13: Contacto vía WhatsApp e Información Institucional

Actor(es)	Invitado / Cliente
Descripción	Canal de atención directa mediante mensajería instantánea y presentación de la marca.
Precondiciones	Ninguna.
Flujo Básico	1. El botón flotante abre el enlace directo configurado en VITE_WHATSAPP_URL. 2. La ruta /sobre-nosotros presenta la historia de la empresa, marcas asociadas y líneas de productos; incluye llamados a la acción (CTAs) dirigidos al catálogo.
Flujos Alternativos / Excepciones	No aplica.
HU Asociada	HU-23

CU-14: Validar Autenticación de Dos Factores (2FA)

Actor(es)	Cliente (en flujo 2FA)
Descripción	Confirmación de identidad mediante código de 6 dígitos enviado por email y gestión de ajuste en perfil.
Precondiciones	Iniciar sesión en una cuenta con la opción two_factor_enabled activa (habilitada por defecto).

Flujo Básico	1. Se genera un código hash de 6 dígitos (válido por 5 min, un solo uso) enviado al correo electrónico. 2. La vista /verificar-2fa valida el código ingresado y retorna los tokens definitivos de sesión. 3. Desde el perfil de usuario se permite activar/desactivar la función (invalidando códigos pendientes).
Flujos Alternativos / Excepciones	Estado 400 si el código es incorrecto, expiró o superó 5 intentos; estado 429 en reenvíos antes de 60 s; cancelar login invalida el proceso.
HU Asociada	HU-03, HU-04

CU-15: Recuperar y Restablecer Contraseña

Actor(es)	Cliente
Descripción	Recuperación de acceso a la cuenta mediante un enlace con token temporal enviado por correo.
Precondiciones	Cuenta registrada previa (la respuesta responde siempre 200 por confidencialidad).
Flujo Básico	1. Vista /olvide-contrasena genera un token SHA-256 (válido 24 horas, un solo uso) enviado vía ZohoMail SMTP. 2. Vista /reestablecer-contrasena asigna la nueva clave (coincidentes, longitud ≥ 6) e inhabilita el token consumido.
Flujos Alternativos / Excepciones	Estado 400 por token inválido, expirado, reutilizado o cuando las contraseñas ingresadas no coinciden.
HU Asociada	HU-05, HU-06

CU-16: Suscribirse a Alertas de Stock ("Avísame")

Actor(es)	Cliente Registrado
Descripción	Notificación automática al usuario cuando un producto actualmente agotado vuelve a tener existencias.
Precondiciones	Sesión de usuario activa; producto seleccionado en estado agotado.

Flujo Básico	1. La opción tipo toggle en la ficha de detalle crea o remueve la alerta (restringida a una por par producto/usuario). 2. Al registrar nuevo stock desde el módulo administrativo, el sistema genera automáticamente una notificación producto_stock y un correo de aviso.
Flujos Alternativos / Excepciones	No aplica.
HU Asociada	HU-14

CU-17: Registrar y Consultar Reseñas de Productos

Actor(es)	Cliente Registrado (para crear) / Todos los usuarios (para consultar)
Descripción	Evaluación y comentarios de 1 a 5 estrellas restringidos a usuarios con compra verificada.
Precondiciones	Para publicar una reseña: contar con una compra entregada del producto y no haberlo reseñado previamente.
Flujo Básico	1. Consulta pública en la ficha de detalle mostrando el promedio ponderado y conteo de opiniones. 2. Creación de reseña validando la compra histórica previa del usuario; la administradora puede supervisar en /admin/resenas.
Flujos Alternativos / Excepciones	Estado 400 por calificación fuera de rango (1 a 5) o reseña duplicada; estado 403 si el usuario no ha comprado el artículo.
HU Asociada	HU-15

CU-18: Gestionar Marcas y Categorías - CRUD Admin

Actor(es)	Administrador
Descripción	Administración de los catálogos base de marcas y categorías del sistema.
Precondiciones	Sesión activa como administrador.

Flujo Básico	1. Operaciones CRUD en las vistas /admin/marcas y /admin/categorias; permite vincular logos por URL o subida de archivo (ImageKit en /marcas). 2. Valida la unicidad de los nombres ingresados.
Flujos Alternativos / Excepciones	Estado 400 por campos requeridos vacíos, nombres duplicados o formatos de imagen no soportados.
HU Asociada	HU-17

CU-19: Consultar Notificaciones In-App

Actor(es)	Cliente Registrado
Descripción	Centro de avisos y novedades accesible desde el ícono de la campana en la barra de navegación.
Precondiciones	Sesión de cliente activa.
Flujo Básico	1. Un proceso de consulta periódica (polling cada 30 s) obtiene las últimas 50 notificaciones y el conteo de no leídas. 2. Permite marcar notificaciones individuales o el lote completo como leídas; al hacer clic redirige al pedido o producto según el atributo datos.
Flujos Alternativos / Excepciones	Sin notificaciones registradas: despliega estado vacío indicando ausencia de novedades.
HU Asociada	HU-20

CU-20: Gestionar Perfil de Usuario

Actor(es)	Cliente Registrado
Descripción	Consulta de datos personales, estado de la sesión y configuración del factor de autenticación 2FA.
Precondiciones	Sesión de usuario activa.

Flujo Básico	1. La vista /perfil despliega la información del cliente (GET /api/auth/perfil) y el conmutador de 2FA. 2. Los tokens JWT expirados se renuevan automáticamente mediante token de refresco; errores 401/422 desloguean y limpian la sesión. 3. La opción de cierre de sesión purga los tokens, datos del carrito y lista de favoritos.
Flujos Alternativos / Excepciones	Ausencia de token de sesión: redirige automáticamente hacia /login.
HU Asociada	HU-22

Repositorio de GitHub

El código desarrollado para el proyecto se aloja en los siguientes repositorios de GitHub:

<https://github.com/Shejin12/KeiBeauty.git> (backend)

<https://github.com/Jbrod5/KeiBeauty-frontend> (frontend)